

DIGITAL ELECTRONICS AND LOGIC DESIGN

SEMESTER 3 - B.TECH KTU - 2024 SCHEME
GAEST305 (COMMON FOR GROUP A BRANCHES)



SCAN TO PURCHASE
WITH FREE SHIPPING



SCAN TO PURCHASE
FROM AMAZON

CHAPTER 4 DATABASE DESIGN THEORY & NORMALIZATION

4.1 DATABASE DESIGN

Database Design can be defined as a set of procedures or collection of tasks involving various steps taken to implement a database. A good database design is important because it helps users to get the right information when he/she needs it. Following are some critical points to keep in mind to achieve a good database design.

1. Data consistency and integrity must be maintained
2. Low Redundancy
3. Faster searching through indices
4. Security measures should be taken by enforcing various integrity constraints.
5. Data should be stored in fragmented bits of information in the most atomic format possible.

What are the Following Steps that can be taken by a Database Designer to Ensure Good Database Design?

Step 1: Determine the goal of your database, and ensure clear communication with the stakeholders (if any). Understanding the purpose of a database will help in thinking of various use cases & where the problem may arise & how we can prevent it.

Step 2: List down all the entities that will be present in the database & what relationships exist among them.

Step 3: Organize the information into different tables such that no or very little redundancy is there.

Step 4: Ensure uniqueness in every table. The uniqueness of records present in any relation is a very crucial part of database design that helps us avoid redundancy. Identify the key attributes to uniquely identify every row from columns. Various key constraints can be used to ensure the uniqueness of the table, also keep in mind the uniquely identifying records must consume as little space as possible & shall not contain any NULL values.

Step 5: After all the tables are structured, and information is organized apply Normalization Forms to identify anomalies that may arise & redundancy that can cause inconsistency in the database.

Primary Terminologies Used in Database Design

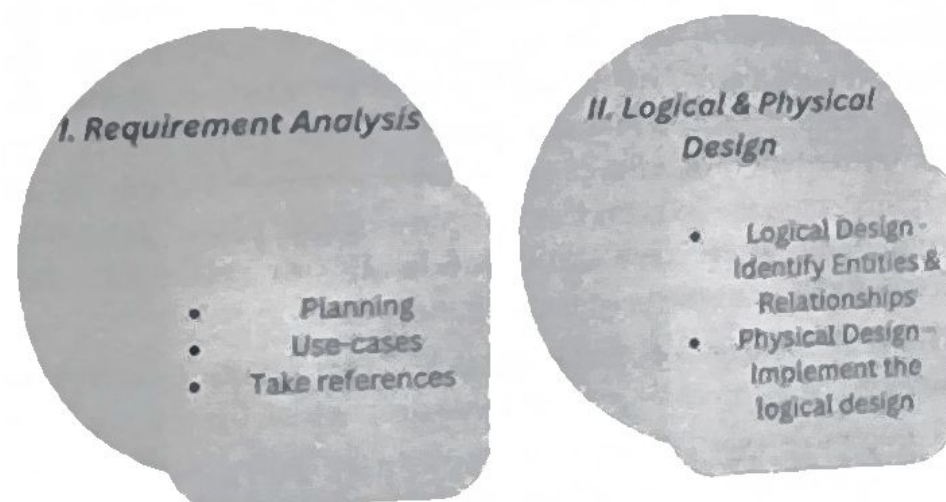
Following are the terminologies that a person should be familiar with before designing a database:

- **Redundancy:** Redundancy refers to the duplicity of the data. There can be specific use cases when we need or don't need redundancy in our Database. For ex: If we have a banking system application then we may need to strictly prevent redundancy in our Database.
- **Schema:** Schema is a logical container that defines the structure & manages the organization of the data stored in it. It consists of rows and columns having data types for each column.
- **Records/Tuples:** A Record or a tuple is the same thing, basically its where our data is stored inside a table
- **Indexing:** Indexing is a data structure technique to promote efficient retrieval of the data stored in our database.
- **Data Integrity & Consistency:** Data integrity refers to the quality of the information stored in our database and consistency refers to the correctness of the data stored.
- **Data Models:** Data models provide us with visual modeling techniques to visualize the data & the relationship that exists among those data. Ex: model, Network Model, Object Oriented Model, Hierarchical model, etc.

- **Normalization:** The process of organizing data to reduce redundancy and dependency by dividing larger tables into smaller ones and defining relationships. It ensures data storage and consistency.
- **Functional Dependency:** Functional Dependency is a relationship between two attributes of the table that represents that the value of one attribute can be determined by another Ex: $\{A \rightarrow B\}$, A & B are two attributes and attribute A can uniquely determine the value of B.
- **Transaction:** Transaction is a single logical unit of work. It signifies that some changes are made in the database. A transaction must satisfy the ACID or BASE properties (depending on the type of Database).
- **Schedule:** Schedule defines the sequence of transactions in which they're executed by one or multiple users.
- **Concurrency:** Concurrency refers to allowing multiple transactions to operate simultaneously without interfering with one another.
- **Constraints:** Constraints are the rules applied to fields in a table to enforce data integrity. e.g., NOT NULL, UNIQUE, CHECK, etc. It ensures data quality and accuracy.

Database Design Lifecycle

The database design lifecycle is as follows:



III. Implementation & Testing

- Test For Integrity & Consistency
- Good to go!

1. Requirement Analysis

It's very crucial to understand the requirements of the application so that user can think in productive terms. And imply appropriate integrity constraints to maintain the data integrity & consistency.

2. Logical & Physical Design

This is the actual design phase that involves various steps that are to be taken while designing a database. This phase is further divided into two stages:

- **Logical Data Model Design:** This phase consists of coming up with a high-level design of the database based on initially gathered requirements to structure & organize the data accordingly. A high-level overview on paper is made of the database without considering the physical level design, this phase proceeds by identifying the kind of data to be stored and what relationship will exist among those data. Entity, Key attributes identification & what constraints are to be implemented is the core functionality of this phase. It involves techniques such as Data Modeling to visualize data, normalization to prevent redundancy, etc.
- **Physical Design of Data Model:** This phase involves the implementation of the logical design made in the previous stage. All the relationships among data and integrity constraints are implemented to maintain consistency & generate the actual database.

3. Data Insertion and testing for various integrity Constraints

Finally, after implementing the physical design of the database, users can input the data & test the integrity. This phase involves testing the database for its integrity to see if something got left out or, if anything new to add & then integrating it with the desired application.

4.2 FUNCTIONAL DEPENDENCIES

Functional dependency (FD) is a set of constraints between two attributes in a relation. Functional dependency says that if two tuples have the same values for attributes $A_1, A_2, \dots, A_n$, then those two tuples must have the same values for attributes $B_1, B_2, \dots, B_n$. Functional dependency is represented by an arrow sign ($\rightarrow$) that is, $X \rightarrow Y$, where X functionally determines Y . The left-hand side attributes determine the values of attributes on the right-hand side.

Armstrong's Axioms

If F is a set of functional dependencies then the closure of F , denoted as F^+ , is the set of all functional dependencies logically implied by F . Armstrong's Axioms are a set of rules that when applied repeatedly, generates a closure of functional dependencies.

- **Reflexive rule** – If α is a set of attributes and β is a subset of α , then α holds β .
- **Augmentation rule** – If $A \rightarrow B$ holds and Y is an attribute set, then $AY \rightarrow BY$ also holds. That is, adding attributes in dependencies, does not change the basic dependencies.
- **Transitivity rule** – Same as transitive rule in algebra, if $A \rightarrow B$ holds and $B \rightarrow C$ holds, then $A \rightarrow C$ also holds. $A \rightarrow B$ is called as a functionally that determines B .

Trivial Functional Dependency

- **Trivial** – If a Functional Dependency (FD) $X \rightarrow Y$ holds, where Y is a subset of X , then it is called a trivial FD. Trivial FDs always hold.
- **Non-trivial** – If an FD $X \rightarrow Y$ holds, where Y is not a subset of X , then it is called a non-trivial FD.
- **Completely non-trivial** – If an FD $X \rightarrow Y$ holds, where X intersect $Y = \emptyset$, it is said to be a completely non-trivial FD.

4.3 NORMALIZATION

Normalization is an important process in database design that helps improve the database's efficiency, consistency, and accuracy. It makes it easier to manage and maintain the data and ensures that the database is adaptable to changing business needs. It is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data but at different places). Normalization generally involves splitting a table into multiple ones which must be linked each time a query is made requiring data from the split tables.

Need of Normalization

The primary objective for normalizing the relations is to eliminate the below anomalies. Failure to reduce anomalies results in data redundancy, which may threaten data integrity and cause additional issues as the database increases. Normalization consists of a set of procedures that assist users in developing an effective database structure.

- **Insertion Anomalies:** It occurs when it is not possible to insert data into a database because the required fields are missing or because the data is incomplete. For example, if a database requires that every record has a primary key, but no value is provided for a particular record, it cannot be inserted into the database.
- **Deletion anomalies:** It occurs when deleting a record from a database and can result in the unintentional loss of data. For example, if a database contains information about customers and orders, deleting a customer record may also delete all the orders associated with that customer.
- **Updation anomalies:** It occurs when modifying data in a database and can result in inconsistencies or errors. For example, if a database contains information about employees and their salaries, updating an employee's salary in one record but not in all related records could lead to incorrect calculations and reporting.

4.3.1 First Normal Form

It is defined in the definition of relations (tables) itself. This rule defines that all the attributes in a relation must have atomic domains. The values in an atomic domain are indivisible units.

Course	Content
Programming	Java, c++
Web	HTML, PHP, ASP

We re-arrange the relation (table) as below, to convert it to First Normal Form.

Course	Content
Programming	Java
Programming	c++
Web	HTML
Web	PHP
Web	ASP

Each attribute must contain only a single value from its pre-defined domain.

4.3.2 Second Normal Form

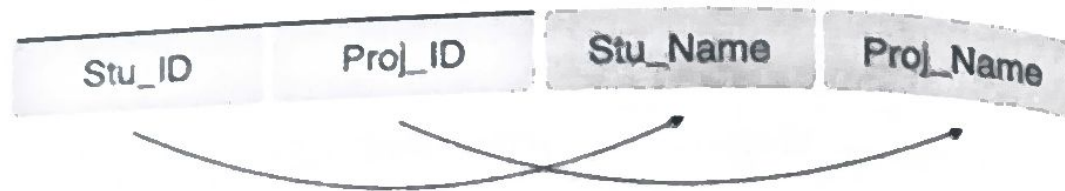
Before learning about the second normal form, we need to understand the following

- **Prime attribute** - An attribute, which is a part of the candidate-key, is known as a prime attribute.
- **Non-prime attribute** - An attribute, which is not a part of the prime-key(prime attribute), is said to be a non-prime attribute.

For a relation to be in Second Normal Form, it must be in First Normal Form and the following must be satisfied.

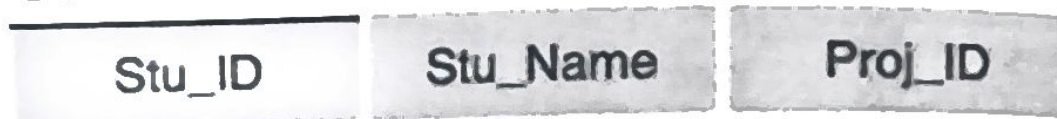
If we follow the Second Normal Form, then every non-prime attribute should be fully functionally dependent on the prime key attribute. That is, if $X \rightarrow A$ holds, then there should not be any proper subset Y of X , for which $Y \rightarrow A$ also holds true.

Student_Project



We see here in Student_Project relation that the prime key attributes are Stu_ID and Proj_ID. According to the rule, non-key attributes, i.e. Stu_Name and Proj_Name must be dependent upon both and not on any of the prime key attributes individually. But we find that Stu_Name can be identified by Stu_ID and Proj_Name can be identified by Proj_ID independently. This is called partial dependency, which is not allowed in Second Normal Form.

Student



Project



We broke the relationship in two as depicted in the above picture. So there exists no partial dependency.

4.3.3 Third Normal Form

For a relation to be in Third Normal Form, it must be in Second Normal form and the following must satisfy

- No non-prime attribute is transitively dependent on the prime key attribute.

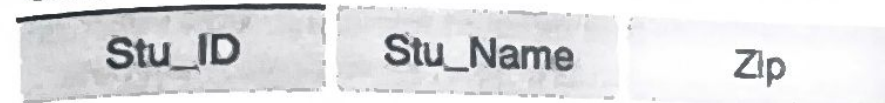
- For any non-trivial functional dependency, $X \rightarrow A$, then either X is a superkey or, A is prime attribute.

Student_Detail



We find that in the above Student_detail relation, Stu_ID is the key and only prime key attribute. We find that City can be identified by Stu_ID as well as Zip itself. Neither Zip is a superkey nor is City a prime attribute. Additionally, $Stu_ID \rightarrow Zip \rightarrow City$, so there exists transitive dependency. To bring this relation into third normal form, we break the relation into two relations as follows

Student_Detail



ZipCodes



Boyce-Codd Normal Form

Boyce-Codd Normal Form (BCNF) is an extension of Third Normal Form on strict terms. BCNF states that

For any non-trivial functional dependency, $X \rightarrow A$, X must be a super-key.

In the above image, Stu_ID is the super-key in the relation Student_Detail and Zip is the super-key in the relation ZipCodes. So, $Stu_ID \rightarrow Stu_Name$, $Zip \rightarrow City$ and $Zip \rightarrow City$

which confirms that both the relations are in BCNF.

Advantages of Normalization

- Normalization eliminates data redundancy and ensures that each piece of data is stored in only one place, reducing the

risk of data inconsistency and making it easier to maintain data accuracy.

- By breaking down data into smaller, more specific tables, normalization helps ensure that each table stores only relevant data, which improves the overall data integrity of the database.
- Normalization simplifies the process of updating data, as it only needs to be changed in one place rather than in multiple places throughout the database.
- Normalization enables users to query the database using a variety of different criteria, as the data is organized into smaller, more specific tables that can be joined together as needed.
- Normalization can help ensure that data is consistent across different applications that use the same database, making it easier to integrate different applications and ensuring that all users have access to accurate and consistent data.

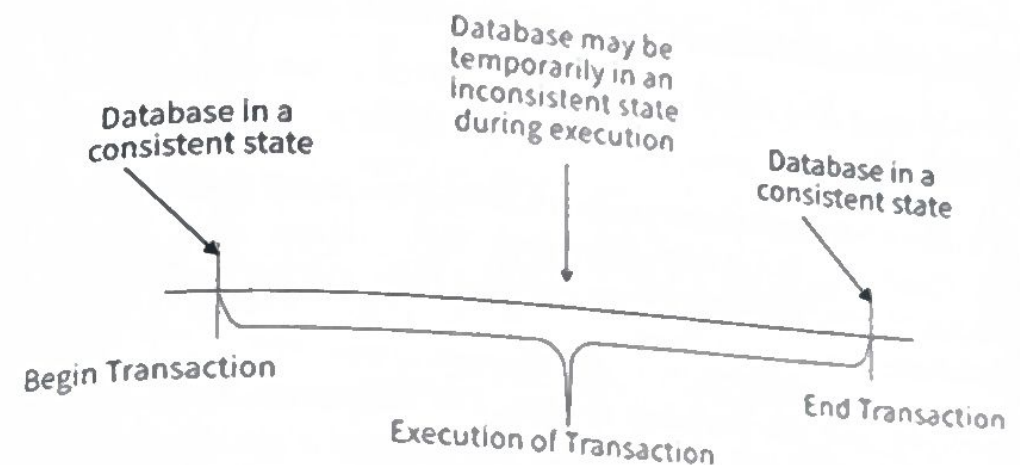
Disadvantages of Normalization

- Normalization can result in increased performance overhead due to the need for additional join operations and the potential for slower query execution times.
- Normalization can result in the loss of data context, as data may be split across multiple tables and require additional joins to retrieve.
- Proper implementation of normalization requires expert knowledge of database design and the normalization process.
- Normalization can increase the complexity of a database design, especially if the data model is not well understood or if the normalization process is not carried out correctly.

4.4 TRANSACTION MANAGEMENT

A transaction refers to a sequence of one or more operations (such as read, write, update, or delete) performed on the database as a single logical unit of work. A transaction ensures that either all the operations are successfully executed (committed) or none of them

take effect (rolled back). Transactions are designed to maintain the integrity, consistency and reliability of the database, even in the case of system failures or concurrent access.



All types of database access operations which are held between the beginning and end transaction statements are considered as a single logical transaction. During the transaction the database is inconsistent. Only once the database is committed the state is changed from one consistent state to another.

Example: Let's consider an online banking application:

Transaction: When a user performs a money transfer, several operations occur, such as:

- Reading the account balance of the sender.
- Writing the deducted amount from the sender's account.
- Writing the added amount to the recipient's account.

4.4.1 Operations of Transaction

A user can make different types of requests to access and modify the contents of a database. So, we have different types of operations relating to a transaction. They are discussed as follows:

1) Read(X)

A read operation is used to read the value of a particular database element X and store it in a temporary buffer in the main memory for further actions such as displaying that value.

Example: For a banking system, when a user checks their balance, a Read operation is performed on their account balance:

```
SELECT balance FROM accounts WHERE account_id = 'A123';
```

This updates the balance of the user's account after withdrawal.

2) Write(X)

A write operation stores updated data from main memory back to the database. It usually follows a read, where data is fetched, modified (e.g., arithmetic changes) and then written back to save the updated value.

Example: For the banking system, if a user withdraws money, a Write operation is performed after the balance is updated:

```
UPDATE accounts SET balance = balance - 100 WHERE account_id = 'A123';
```

This updates the balance of the user's account after withdrawal.

3) Commit

This operation in transactions is used to maintain integrity in the database. Due to some failure of power, hardware, or software, etc., a transaction might get interrupted before all its operations are completed. This may cause ambiguity in the database, i.e. it might get inconsistent before and after the transaction.

Example: After a successful money transfer in a banking system, a Commit operation finalizes the transaction:

```
COMMIT;
```

Once the transaction is committed, the changes to the database are permanent and the transaction is considered successful.

4) Rollback

A rollback undoes all changes made by a transaction if an error occurs, restoring the database to its last consistent state. It helps prevent data inconsistency and ensures safety.

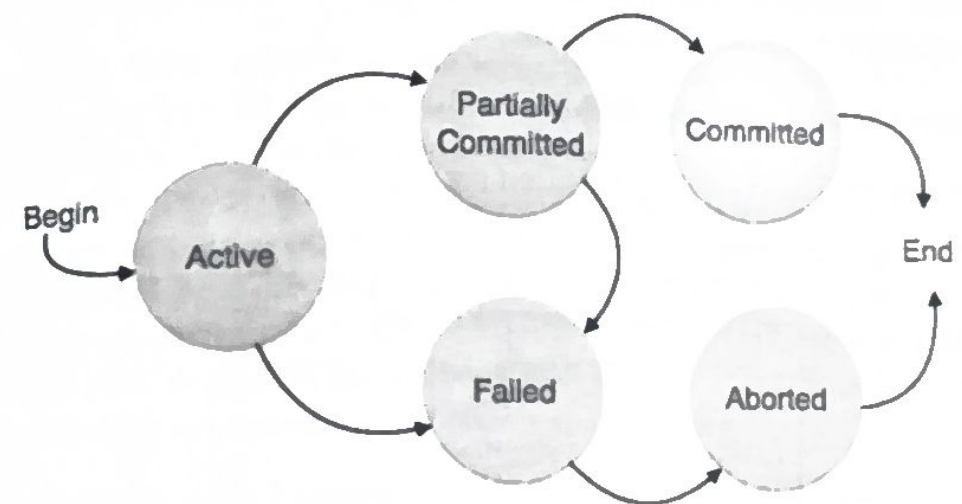
Example: Suppose during the money transfer process, the system encounters an issue, like insufficient funds in the sender's account. In that case, the transaction is rolled back:

```
ROLLBACK;
```

This will undo all the operations performed so far and ensure that the database remains consistent.

4.4.2 States of Transactions

A transaction in a database can be in one of the following states



- **Active** – In this state, the transaction is being executed. This is the initial state of every transaction.
- **Partially Committed** – When a transaction executes its final operation, it is said to be in a partially committed state.
- **Failed** – A transaction is said to be in a failed state if any of the checks made by the database recovery system fails. A failed transaction can no longer proceed further.
- **Aborted** – If any of the checks fails and the transaction has reached a failed state, then the recovery manager rolls back all its write operations on the database to bring the database back to its original state where it was prior to the execution of the transaction. Transactions in this state are called aborted. The database recovery module can select one of the two operations after a transaction aborts
 - Re-start the transaction
 - Kill the transaction

- **Committed** – If a transaction executes all its operations successfully, it is said to be committed. All its effects are now permanently established on the database system.

4.4.3 Problems and failures in transaction

Failure in terms of a database can be defined as its inability to execute the specified transaction or loss of data from the database. A DBMS is vulnerable to several kinds of failures and each of these failures needs to be managed differently. There are many reasons that can cause database failures such as network failure, system crash, natural disasters, carelessness, sabotage (corrupting the data intentionally), software errors, etc.

Failure Classification in DBMS

A failure in DBMS can be classified as:

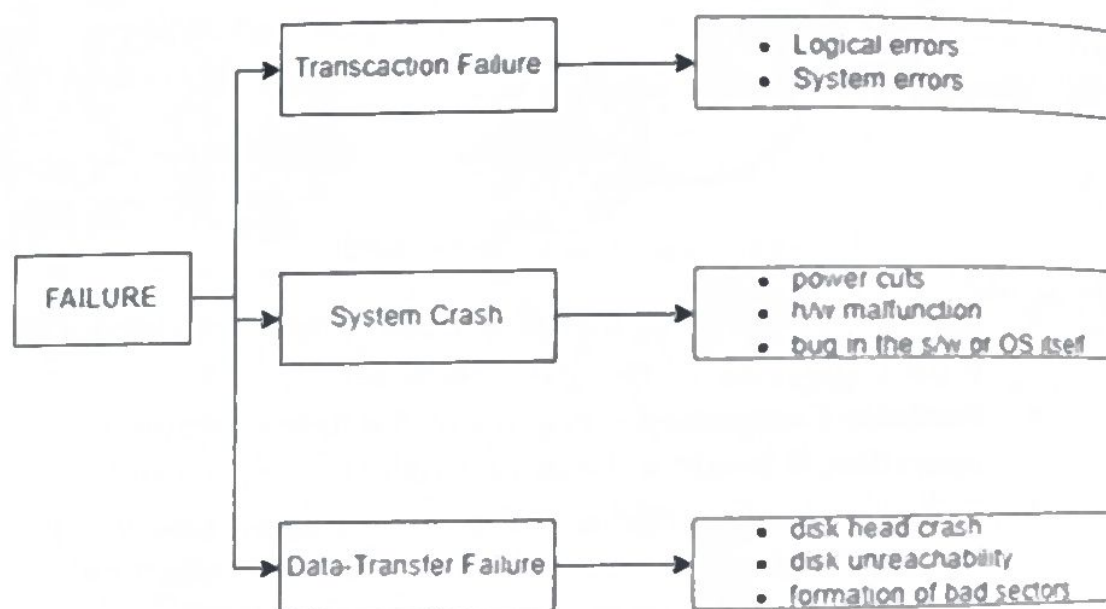


Fig: Failure Classification in DBMS

- **Transaction Failure:**

If a transaction is not able to execute or it comes to a point from where the transaction becomes incapable of executing further then it is termed as a failure in a transaction.

Reason for a transaction failure in DBMS:

1. **Logical error:** A logical error occurs if a transaction is unable to execute because of some mistakes in the code or due to the presence of some internal faults.
2. **System error:** Where the termination of an active transaction is done by the database system itself due to some system issue or because the database management system is unable to proceed with the transaction. *For example-* The system ends an operating transaction if it reaches a deadlock condition or if there is an unavailability of resources.

- **System Crash:**

A system crash usually occurs when there is some sort of hardware or software breakdown. Some other problems which are external to the system and cause the system to abruptly stop or eventually crash include failure of the transaction, operating system errors, power cuts, main memory crash, etc. These types of failures are often termed soft failures and are responsible for the data losses in the volatile memory. It is assumed that a system crash does not have any effect on the data stored in the non-volatile storage and this is known as the fail-stop assumption.

- **Data-transfer Failure:**

When a disk failure occurs amid data-transfer operation resulting in loss of content from disk storage then such failures are categorized as data-transfer failures. Some other reasons for disk failures include disk head crash, disk unreachability, formation of bad sectors, read-write errors on the disk, etc. In order to quickly recover from a disk failure caused amid a data-transfer operation, the backup copy of the data stored on other tapes or disks can be used. Thus it's a good practice to backup your data frequently.

ACID Properties of Transaction

Transactions in DBMS must ensure data is accurate and reliable. They follow four key ACID properties:

1. **Atomicity:** A transaction is all or nothing. If any part fails, the entire transaction is rolled back. Example: While transferring money, both debit and credit must succeed. If one fails, nothing should change.

2. **Consistency:** A transaction must keep the database in a valid state, moving it from one consistent state to another. Example: If the balance is ₹1000 and ₹200 is withdrawn, the new balance should be ₹800.
3. **Isolation:** Transactions run independently. One transaction's operations should not affect another's intermediate steps. Example: Two users withdrawing from the same account must not interfere with each other's balance updates.
4. **Durability:** Once a transaction is committed, its changes stay even if the system crashes. Example: After a successful transfer, the updated balance remains safe despite a power failure.

4.4.4 Characterizing schedules based on recoverability and serializability

Scheduling is the process of determining the order in which transactions are executed. When multiple transactions run concurrently, scheduling ensures that operations are executed in a way that prevents conflicts or overlaps between them.

There are several types of schedules, all of them are depicted in the diagram below:

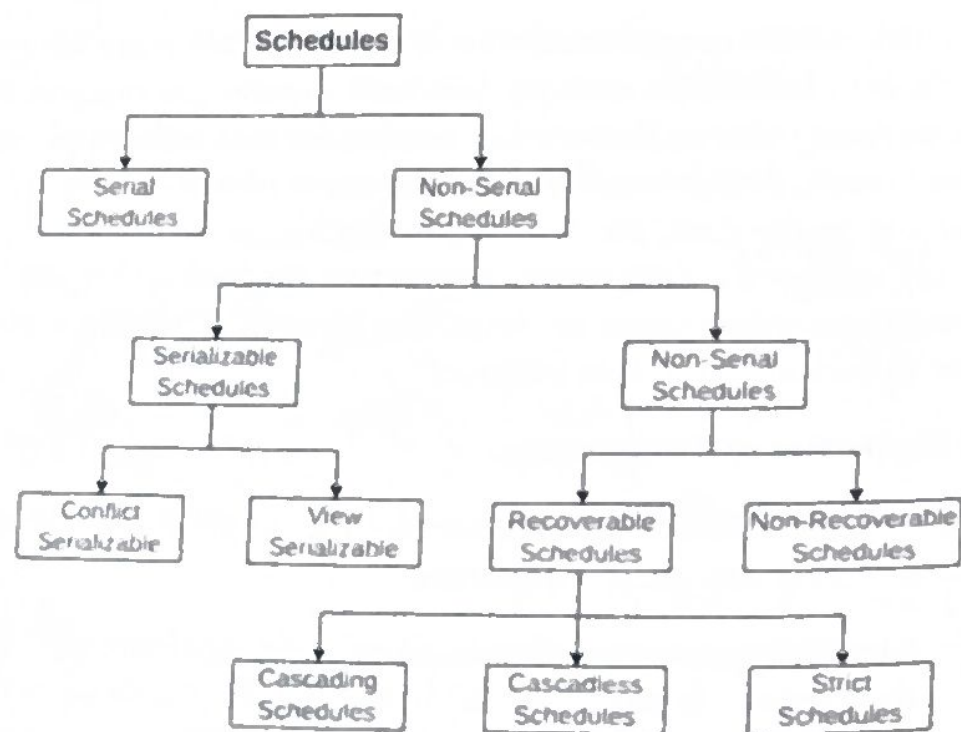


Fig: Types of schedules in DBMS

• Serial Schedule

Schedules in which the transactions are executed non-interleaved, i.e., a serial schedule is one in which no transaction starts until a running transaction has ended are called serial schedules.

Example: Consider the following schedule involving two transactions T1 and T2.

T1	T2
R(A)	
W(A)	
R(B)	
	W(B)
	R(A)
	R(B)

where R(A) denotes that a read operation is performed on some data item 'A'. This is a serial schedule since the transactions perform serially in the order T1 → T2.

• Non-Serial Schedule

This is a type of Scheduling where the operations of multiple transactions are interleaved. This might lead to a rise in the concurrency problem. The transactions are executed in a non-serial manner, keeping the end result correct and the same as the serial schedule. This sort of schedule does not provide any benefit of the concurrent transaction. It can be of two types namely,

1) Serializable Scheduling (Concurrency Control)

- Ensures database consistency in non-serial schedules by verifying if they behave like serial schedules.
- In a serial schedule, transactions execute one after another, ensuring correctness without conflicts.
- Non-serial schedules allow concurrent transactions but must be serializable to maintain correctness.

Why Serializable?

- Prevents anomalies due to concurrent execution.
- Allows better CPU and resource utilization.
- Improves throughput without sacrificing consistency

Two types of serializable scheduling are:

i. **Conflict Serializable:** A schedule is called conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations. Two operations are said to be conflicting if all conditions satisfy:

- They belong to different transactions
- They operate on the same data item
- At least one of them is a write operation

ii. **View Serializable:** A Schedule is called view serializable if it is view equal to a serial schedule (no overlapping transactions). A conflict schedule is a view serializable but if the serializability contains blind writes, then the view serializable does not conflict serializable.

2) Non-Serializable Scheduling

Schedules that do not preserve serial equivalence and may lead to inconsistencies if not handled carefully.

i. **Recoverable Schedule:** Schedules in which transactions commit only after all transactions whose changes they read commit are called recoverable schedules. In other words, if some transaction T_j is reading, value updated or written by some other transaction T_i , then the commit of T_j must occur after the commit of T_i .

Example: Consider the following schedule involving two transactions T_1 and T_2 .

T1	T2
R(A)	
W(A)	
	W(A)
	R(A)
Commit	
	Commit

This is a recoverable schedule since T_1 commits before T_2 , that makes the value read by T_2 correct. There can be three types of recoverable schedule:

- **Cascading Schedule:** Also called Avoids Cascading Aborts/rollbacks (ACA). When there is a failure in one dependent transactions, then such scheduling is referred to as Cascading rollback or cascading abort.

Example:

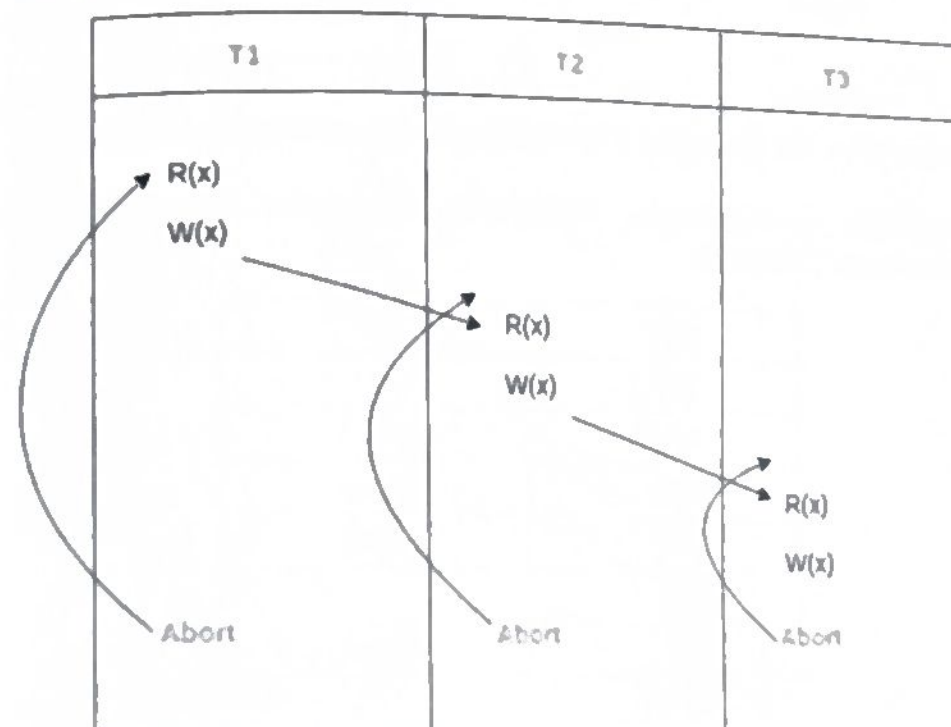


Fig: Cascading Abort

- **Cascadeless Schedule:** Schedules in which transactions read values only after all transactions whose changes they are going

to read commit are called cascadeless schedules. Avoids that a single transaction abort leads to a series of transaction rollbacks.

A strategy to prevent cascading aborts is to disallow a transaction from reading uncommitted changes from another transaction in the same schedule. In other words, if some transaction T_j wants to read a value updated or written by some other transaction T_i , then the commit of T_j must read it after the commit of T_i .

Example1: Consider the following schedule involving two transactions T_1 and T_2 .

T1	T2
R(A)	
W(A)	
	W(A)
commit	
	R(A)
	commit

This schedule is cascadeless. Since the updated value of A is read by T_2 only after the updating transaction i.e. T_1 commits.

Example2: Consider the following schedule involving two transactions T_1 and T_2 .

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
abort	
	abort

It is a recoverable schedule but it does not avoid cascading aborts. It can be seen that if T_1 aborts, T_2 will have to be aborted too in order to maintain the correctness of the schedule as T_2 has already read the uncommitted value written by T_1 .

- Strict Schedule:** A schedule is strict if for any two transactions T_i, T_j , if a write operation of T_i precedes a conflicting operation of T_j (either read or write), then the commit or abort event of T_i also precedes that conflicting operation of T_j . In other words, T_j can read or write updated or written value of T_i only after T_i commits/aborts.

Example: Consider the following schedule involving two transactions T_1 and T_2 .

T1	T2
R(A)	
	R(A)
W(A)	
commit	
	W(A)
	R(A)
	commit

This is a strict schedule since T_2 reads and writes A which is written by T_1 only after the commit of T_1 .

ii. Non-Recoverable Schedule:

Example: Consider the following schedule involving two transactions T_1 and T_2 .

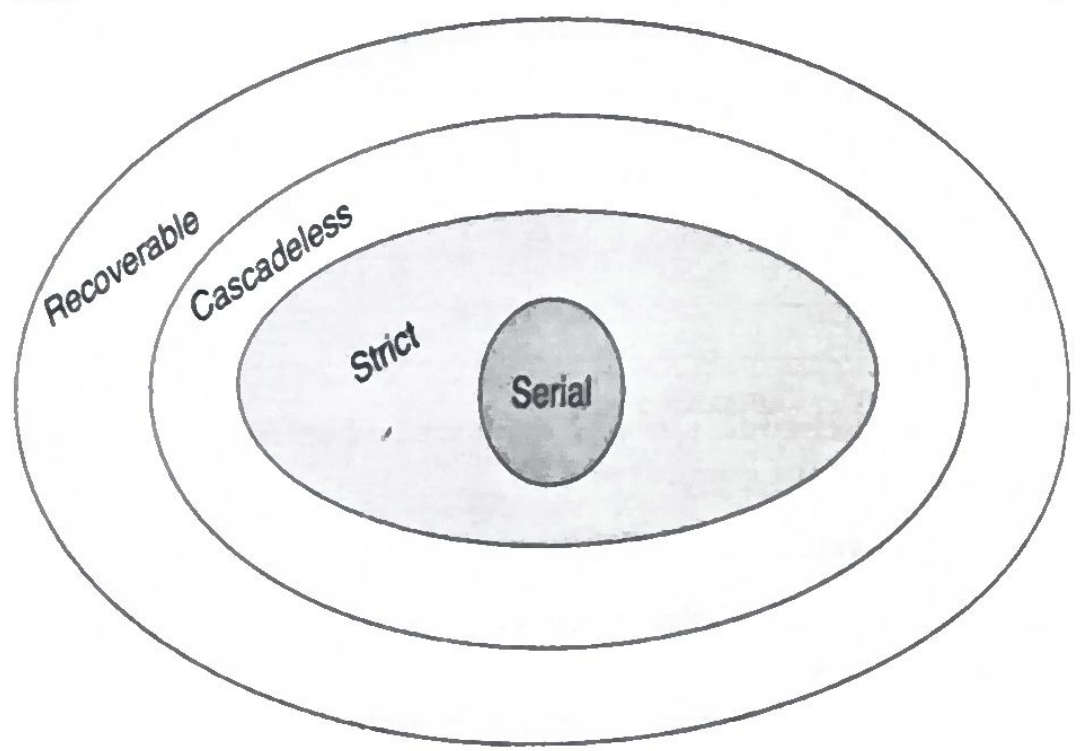
T1	T2
R(A)	
W(A)	
	W(A)
	R(A)
	commit
abort	

T_2 read the value of A written by T_1 , and committed. T_1 later aborted, therefore the value read by T_2 is wrong, but since T_2 committed, this schedule is non-recoverable.

Note - It can be seen that:

- Cascadeless schedules are stricter than recoverable schedules or are a subset of recoverable schedules.
- Strict schedules are stricter than cascadeless schedules or are a subset of cascadeless schedules.
- Serial schedules satisfy constraints of all recoverable, cascadeless and strict schedules and hence is a subset of strict schedules.

The relation between various types of schedules can be depicted as:



Example:

Consider the following schedule:

S: R1(A), W2(A), Commit2, W1(A), W3(A), Commit3, Commit1

Question: Which of the following is true?

- (A) The schedule is view serializable schedule and strict recoverable schedule
- (B) The schedule is non-serializable schedule and strict recoverable schedule
- (C) The schedule is non-serializable schedule and is not strict recoverable schedule.

(D) The Schedule is serializable schedule and is not strict recoverable schedule

Solution: The schedule can be re-written as:-

T1	T2	T3
R(A)		
	W(A)	
	commit	
W(A)		
		W(A)
		commit
commit		

First of all, it is a view serializable schedule as it has a view equal serial schedule T1 → T2 → T3 which satisfies the initial and updated reads and final write on variable A which is required for view serializability. Now we can see there is write - write pair done by transactions T1 followed by T3 which is violating the above-mentioned condition of strict schedules as T3 is supposed to do write operation only after T1 commits which is violated in the given schedule. Hence the given schedule is serializable but not strict recoverable. So, option (D) is correct.

Advantages of Serializability

- **Predictable Execution:** Transactions behave consistently without surprises – no data loss or corruption.
- **Easier Debugging:** Each transaction runs independently, making errors easier to trace and fix.
- **Lower Costs:** Reduces need for complex hardware and can cut down development costs.
- **Better Performance:** Optimized, consistent execution can sometimes outperform non-serial schedules.

4.5 CONCURRENCY CONTROL WITH TWO-PHASE LOCKING TECHNIQUES

The Two-Phase Locking (2PL) Protocol is a key technique used in DBMS to manage how multiple concurrent transactions access and modify data. When many users or processes interact with a database, it's important to ensure that data remains consistent and error-free. Without proper management, issues like data conflicts or corruption can occur.

The Two-Phase Locking Protocol resolves this issue by defining clear rules for managing data locks. It divides a transaction into two phases:

1. **Growing Phase:** In this step, the transaction gathers all the locks it needs to access the required data. During this phase, it cannot release any locks.
2. **Shrinking Phase:** Once a transaction starts releasing locks, it cannot acquire any new ones. This ensures that no other transaction interferes with the ongoing process.

Types of Lock

- **Shared Lock (S):** Shared Lock is also called a read-only lock, allows multiple transactions to access the same data item for reading at the same time. However, transactions with this lock cannot make changes to the data. A shared lock is requested using the lock-S instruction.
- **Exclusive Lock (X):** An Exclusive Lock allows a transaction to both read and modify a data item. This lock is exclusive, meaning no other transaction can access the same data item while this lock is held. An exclusive lock is requested using the lock-X instruction.

Lock Conversions

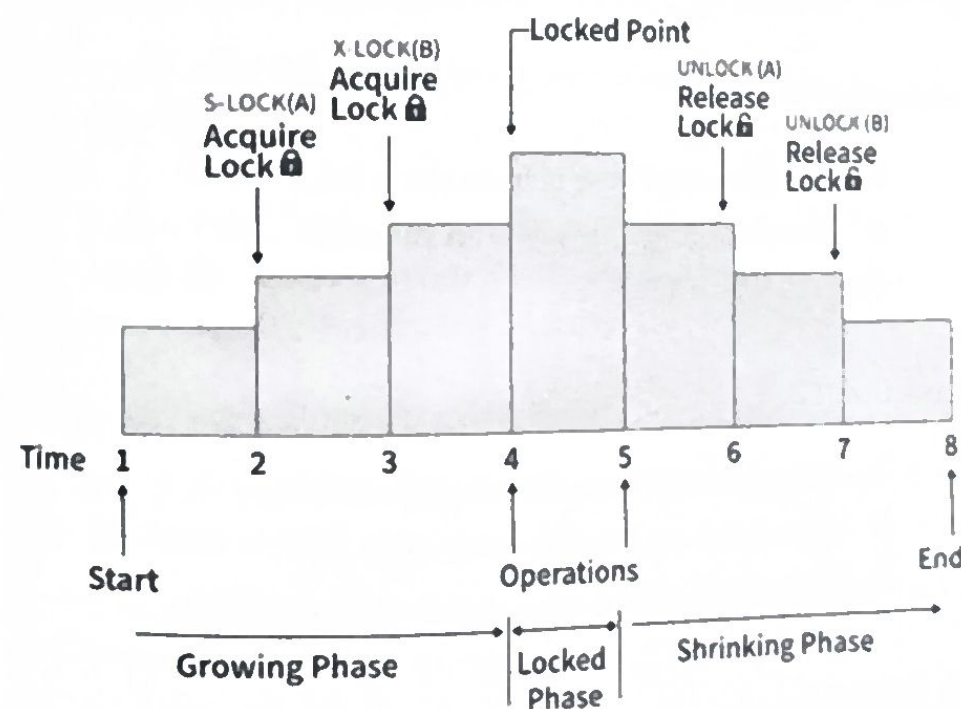
In the Two-Phase Locking Protocol, lock conversion means changing the type of lock on data while a transaction is happening. This process is carefully controlled to maintain consistency in the database.

1. **Upgrading a Lock:** This means changing a shared lock (S) to an exclusive lock (X). For example, if a transaction initially only needs to read data (S) but later decides it needs to update the same data, it can request an upgrade to an exclusive lock (X). However, this can only happen during the Growing Phase, where the transaction is still acquiring locks.

Example: A transaction reads a value (S lock) but then realizes it needs to modify the value. It upgrades to an X lock during the Growing Phase.

2. **Downgrading a Lock:** This means changing an exclusive lock (X) to a shared lock (S). For instance, if a transaction initially planned to modify data (X lock) but later decides it only needs to read it, it can downgrade the lock. However, this must happen during the Shrinking Phase, where the transaction is releasing locks.

Example: A transaction modifies a value (X lock) but later only needs to read the value, so it downgrades to an S lock during the Shrinking Phase.



These rules ensure that the Two-Phase Locking Protocol maintains consistency and avoids conflicts between transactions. By limiting

when upgrades and downgrades can occur, the system prevents situations where multiple transactions interfere with each other's operations.

Let's see a transaction implementing 2-PL.

	T1	T2
1	lock-S(A)	
2		lock-S(A)
3	lock-X(B)	
4		
5	Unlock(A)	
6		lock-X(C)
7	Unlock(B)	
8		Unlock(A)
9		Unlock(C)
10		

This is a basic outline of a transaction that demonstrates how locking and unlocking work in the Two-Phase Locking Protocol (2PL).

Transaction T1

- The growing Phase is from steps 1-3
- The shrinking Phase is from steps 5-7
- Lock Point at 3

Transaction T2

- The growing Phase is from steps 2-6
- The shrinking Phase is from steps 8-9
- Lock Point at 6

Lock Point

The lock point in a transaction is the moment when the transaction finishes acquiring all the locks it needs. After this point, no new locks can be added, and the transaction starts releasing locks. It's a key

step in the Two-Phase Locking Protocol to ensure the rules of growing and shrinking phases are followed.

Example of 2PL

Imagine a library system where multiple users can borrow or return books. Each action (like borrowing or returning) is treated as a transaction. Here's how the Two-Phase Locking Protocol (2PL) works, including the lock point:

User A wants to:

1. Check the availability of Book X.
2. Borrow Book X if it's available.
3. Update the library's record.

Growing Phase (Locks are Acquired):

1. User A locks Book X with a shared lock (S) to check its availability.
2. After confirming the book is available, User A upgrades the lock to an exclusive lock (X) to borrow it.
3. User A locks the library's record to update the borrowing details.

Lock Point: Once User A has acquired all the necessary locks (on Book X and the library record), the transaction reaches the lock point. No more locks can be acquired after this.

Shrinking Phase (Locks are Released):

1. User A updates the record and releases the lock on the library's record.
2. User A finishes borrowing and releases the exclusive lock on Book X.

This process ensures that no other user can interfere with Book X or the library record during the transaction, maintaining data accuracy and consistency. The lock point ensures that all locks are acquired before any are released, following the 2PL rules.

Drawbacks of 2PL

Two-phase locking (2PL) ensures that transactions are executed in the correct order by using two phases: acquiring and releasing locks. However, it has some drawbacks:

- **Deadlocks:** Transactions can get stuck waiting for each other's locks, causing them to freeze indefinitely.
- **Cascading Rollbacks:** If one transaction fails, others that depend on it might also fail leading to inefficiency and potential data issues.
- **Lock Contention:** Too many transactions competing for the same locks can slow down the system, especially when many users are working at the same time.
- **Limited Concurrency:** The strict rules of 2PL can reduce how many transactions can run at once, resulting in slower performance and longer wait times.

Dirty Read

A Dirty Read in SQL occurs when a transaction reads data that has been modified by another transaction, but not yet committed. In other words, a transaction reads uncommitted data from another transaction, which can lead to incorrect or inconsistent results. This situation can occur when a transaction modifies a data item and then fails to commit the changes due to a system failure, network error, or other issue. If another transaction reads the modified data before the first transaction has a chance to commit, it can lead to a dirty read.

Cascading Rollbacks in 2-PL

Let's see the following Schedule:

	T ₁	T ₂	T ₃
1	Lock X(A)		
2	Read(A)		
3	Write(A)		
4	Lock S(B) → LP		
5	Read(B)		
6	Unlock(A), Unlock(B)		
7		Lock X(A) → LP	
8		Read(A)	
9		Write(A)	
10		Unlock(A)	
11			Lock S(A) → LP
12			Read(A)

LP : Lock Point

Read(A) in T₂ and T₃ denotes Dirty Read because of Write(A) in T₁.

The image illustrates a transaction schedule using the Two-Phase Locking (2PL) protocol, showing the sequence of actions for three transactions T₁, T₂ and T₃.

Key Points:

1. Transaction T₁:

- T₁ acquires an exclusive lock (X) on data item A, performs a write operation on A and then acquires a shared lock (S) on B.
- T₁ reaches its lock point (LP) after acquiring all locks.
- Eventually, T₁ fails and a rollback is triggered, undoing its changes.

2. Transaction T₂:

- T₂ reads A after T₁ writes A. This is called a dirty read because T₁'s write is not committed yet.

- When T1 rolls back, T2's operations become invalid, and it is also forced to rollback.

3. Transaction T3:

- T3 reads A after T2 reads A. Since T2 depends on the uncommitted changes of T1, T3 indirectly relies on T1's changes.
- When T1 rolls back, T3 is also forced to rollback even though it was not directly interacting with T1's operations.

Cascading Rollback Problem:

- Here, T2 and T3 both depend on uncommitted data from T1 (either directly or indirectly).
- When T1 fails and rolls back all dependent transactions (T2 and T3) must also rollback because their operations were based on invalid data.
- Cascading rollbacks waste system resources, reduce concurrency and lead to inefficiency.
- In large systems, this can significantly affect performance and make recovery more complex.

How to avoid it?

- **Strict 2PL:** Ensure that transactions release their locks only after they commit, preventing other transactions from accessing uncommitted changes.
- **Rigorous 2PL:** Extend strict 2PL to hold both read and write locks until the transaction commits, offering even stronger guarantees.

Deadlock in 2-PL

Consider the example below.

We have two transactions T1 and T2.

Schedule: Lock-X1(A) Lock-X2(B) Lock-X1(B) Lock-X2(A)

This sequence represents a locking scenario where two transactions, T1 and T2 are trying to lock two resources, A and B in a particular order. Here's what each step means:

1. **Lock-X1(A):**
Transaction T1 acquires an exclusive lock on resource A. This means T1 has full control over A and no other transaction can use it until T1 releases the lock.
2. **Lock-X2(B):**
Transaction T2 acquires an exclusive lock on resource B. Similarly, T2 now has full control over B and no other transaction can access B until T2 releases the lock.
3. **Lock-X1(B):**
Transaction T1 tries to acquire an exclusive lock on resource B but T2 already holds the lock on B. So, T1 must wait for T2 to release the lock.
4. **Lock-X2(A):**
At the same time, Transaction T2 tries to acquire an exclusive lock on resource A but T1 already holds the lock on A. So, T2 must wait for T1 to release the lock.

The above-mentioned type of 2-PL is called Basic 2PL. These transactions ensure Conflict Serializability but do not prevent Cascading Rollback and Deadlock. To prevent these issues Strict Two Phase Locking and Rigorous Two Phase Locking can be used.

4.6 DATABASE TRANSACTION RECOVERY TECHNIQUES IN DBMS

When a system failure occurs during a transaction (such as power failure, hardware malfunction, software error, or deadlock), the database can become inconsistent or partially updated. Recovery techniques are designed to restore the database to a consistent state, ensuring the ACID properties are maintained.

Checkpointing

- A checkpoint is a snapshot of the database state at a specific point in time.

- During recovery, the system does not have to start from scratch; it can roll back to the last checkpoint and replay committed transactions.

Undo and Redo Operations

- Recovery involves two essential types of operations:
 - **Undo:** Revert the effects of incomplete or uncommitted transactions.
 - **Redo:** Reapply committed transactions that might not have been written to disk at the time of failure.
- The Write-Ahead Logging (WAL) principle ensures that log records describing changes are written to stable storage before the changes themselves are applied to the database.

4.6.1 Deferred Update (No-Undo/Redo Approach)

- In this approach, the system does not write changes to the database immediately.
- Changes are made in temporary memory (buffer).
- Only after a COMMIT is the data written to the database.

4.6.2 Immediate Update (Undo/Redo Approach)

- Changes are written to the database as they happen, even before a COMMIT.
- To ensure recoverability, logs are used to undo incomplete transactions and redo committed ones.

4.6.3 Shadow Paging

- Instead of modifying the actual database pages directly, the system maintains a shadow copy of the database.
- All operations are performed on a copy.
- Once the transaction successfully commits, the new version replaces the old version atomically.

CHAPTER 5 NoSQL CONCEPTS

5.1 INTRODUCTION TO NoSQL CONCEPTS

NoSQL (Not Only SQL) databases are designed to handle large volumes of unstructured and semi-structured data. Unlike traditional relational databases that rely on fixed schemas and tables, NoSQL offers flexible data models and supports horizontal scaling. This makes them well-suited for modern applications that require high performance, scalability, and the ability to manage diverse data types efficiently.

Key Features

- **Dynamic schema:** Allow flexible shaping of data to meet new requirements without the need to migrate or change schemas.
- **Horizontal scalability:** They scale horizontally for adding more nodes into the existing ones and acquire enough storage for even bigger datasets and much higher traffic by distributing the load on multiple servers.
- **Document-based:** Data are presented in flexible, semi-structured formats like JSON/BSON (e.g., MongoDB).
- **Key-value-based:** They possess a simple but fast access pattern (e.g., Redis) by storing data as pairs of keys and values.
- **Column-based:** Data are organized into columns instead of rows (e.g., CASSANDRA).
- **Distributed and high availability:** They are designed to be highly available and to automatically handle node failures and data replication across multiple nodes in a database cluster.